

Технически въпроси — отговори

Кратки, по същество. Решенията препращат към реалните задачи в това геро, където е приложимо.

WordPress и PHP

1. Как подхождаш към WordPress сайт, в който не познаваш кода — какво е първото, което правиш?

Подхождам като към всеки непознат код. Първо правя локално копие на сайта — клонирам файловете и dump на базата (или през миграционен плъгин) — и го пускам в локална среда, за да работя без риск за живия сайт. После гледам конкретно: папките `wp-content/plugins/` и `wp-content/themes/` за активните плъгини и темата, дали има `mu-plugins/`, `child theme` или `custom` код във `functions.php`, и `wp-config.php` за конфигурацията. Включвам `WP_DEBUG_LOG`, за да хващам грешките в `debug.log`, и слагам Query Monitor, за да видя кой плъгин какви `hooks` и заявки пуска. Така разделям кое е стандартен WordPress, кое плъгин и кое нечий `custom` код — и чак тогава променям.

2. Плъгин актуализация е счупила нещо на live сайта. Как дебъгваш? Първо спирам щетата: връщам плъгина към предишната версия (`rollback`) или го деактивирам, за да тръгне сайтът. После възпроизвеждам проблема в локалното копие — включвам `WP_DEBUG_LOG` и гледам `debug.log` за конкретната PHP грешка, а с Query Monitor виждам коя заявка или `hook` гърми. Чета `changelog`-а на плъгина за `breaking changes` (често смяна на `hook` или промяна в API). Поправям в локалното, тествам, и чак тогава пускам на живо. Принципът: на `production` не дебъгвам — първо го стабилизирам, после възпроизвеждам безопасно.

3. Как добавяш нова функционалност без да пипаш production директно? Никога директно на живо. Разработвам в локална среда (или `staging`) под `version control` (`git`) — функционалността е собствен плъгин или в `child theme`, не пипам `core` или `parent` темата. В екип промяната минава като `Pull Request` с ревю и одобрение от тийм лидера (преглед на кода + `preview/staging`), и чак след одобрение се слива на `production`. В моя собствен проект работя сам, затова ползвам `feature branches` + `self-review` + `staging` — дисциплината е същата: нищо не отива на `production` непрегледано и нетествано.

4. Имаш ли опит с Custom Post Types, ACF, REST API? REST API — да, имам реален опит: освен в `pensa.club`, съм писал REST API-та и в над 10 други проекта (по-малки, но реални). CPT и ACF — имам базово разбиране какво са и за какво служат (CPT = собствен тип съдържание; ACF = custom полета през плъгин), но още не съм ги ползвал в реален проект. С `backend/REST` опита ми обаче бих ги усвоил бързо.

5. Платформа с платени членове — страница само за логнат потребител с активен абонамент. Как? Две проверки преди да покаже страницата: дали потребителят е логнат (`is_user_logged_in()`) и дали има активен абонамент (статусът идва от членския плъгин — напр. `WooCommerce Memberships` / `MemberPress` — или от `usermeta/membership API`). Ако някоя не мине — редирект към `login` или `pricing` страницата, или показвам ограничено съдържание. Същата логика — проверка преди достъп — реализирах в Задача А (401 за нелогнат, 403 без активни права).

Next.js и клиентска зона

1. Имаш ли опит с Next.js? Разкажи за проект. Да. Работих по frontend-а на `dentalpoint.bg` — `production` сайт на Next.js 14 (App Router, TypeScript, Tailwind): помогнах на колега с компоненти и UI. Проектът е с интернационализация (`next-intl`, `[locale]` рутиране), автентикация (`next-auth`), оптимизация на изображения (`sharp` + `blur placeholders`) и SEO (`sitemap/robots`) — с функции като галерии, `before/after slider` и `lightbox`. Освен това правих собствени Next.js експерименти — започнах да мигрирам части от моя проект `pensa.club` към Next.js заради SEO (SSR), но решихме SEO-то по друг начин, така че останаха като `proof-of-concept`. Минал съм и курс по Next.js. Основата ми е React (`pensa.club` — голям `production` проект), затова Next.js ми ляга естествено.

2. Клиентската зона чете от WP API. Ако нещо не се зарежда — как локализираш проблема? Изолирам слой по слой, не гадая. Първо извиквам WP REST endpoint-а директно (curl/Postman): ако WordPress не върне коректно → проблемът е там (плъгин, права, самата заявка). Ако върне правилно → WordPress е ок и гледам Next.js слоя: network таба в браузъра (какъв статус връща, CORS, правилен ли е URL-ът), сървърните логове, дали env променливата с API адреса сочи където трябва, и кеширане (стари данни). Така за минути виждам в кой от трите слоя е проблемът.

3. Преместване на Next.js на Vercel — стъпки + какво може да се чуе? Първо се уверявам, че кодът е във version control — ако още няма Git repo, го вкарвам и пушвам в GitHub. После свързвам репото с Vercel, пренасям env променливите в настройките, правя deploy и тествам на preview адреса, без да пипам живия сайт. Чак като е ок, добавям истинския домейн (my.tonkin.bg) и насочвам DNS-а (CNAME) към Vercel.

Какво може да се чуе и как го решавам:

- **Запис на файлове** (serverless няма постоянен диск) → местя ги в облачно хранилище (S3 / R2).
- **Cron задачи** (няма постоянен сървър) → Vercel Cron Jobs.
- **Абсолютни/локални пътища** (средата е различна) → правя ги релативни или през правилните API-та.
- **Env променливи** → server-only отделно от публичните (NEXT_PUBLIC_); тайните остават server-side.
- **Serverless лимити** (timeout/размер) → тежките/дълги операции изнасям извън заявката.
- **Зашити URL-и за стария сървър** → през env променливи, не hardcoded.

(Разписано подробно и в Задача C — т.1.2 и т.4.)

4. Как пазиш API ключове в Next.js — допустимо и забранено? **Забранено:** ключ в кода, в Git, или с NEXT_PUBLIC_ префикс — последното го вгражда в браузърния bundle и го прави публичен. **Допустимо:** ключовете в env (Vercel encrypted store, или .env.local, който е gitignored), използвани само server-side (в API routes / server компоненти, не в браузъра), .env.example с празни стойности в репото, и ротация при съмнение за изтичане.

Claude API / LLMs

1. Как пишеш system prompt за конкретна роля — задължителни елементи? Независимо от продукта, един system prompt за роля включва:

- **идентичност** — кой е асистентът;
- **цел** — какво трябва да постига;
- **аудитория/контекст** — за кого работи (ниво, нужди);
- **тон/стил** на комуникация;
- **ограничения** — какво да избягва (обхват, безопасност, правни/етични граници);
- **формат на отговора** — структура, дължина, език, ако има изисквания;
- **поведение при edge cases** — какво прави при въпрос извън обхвата, неяснота или липса на данни.

При нужда добавям кратки примери (few-shot), за да задам формата/качеството. Динамичните данни за конкретния потребител ги подавам отделно, не зашита в system prompt-а. В Задачи B и D това се вижда приложено за МАБИ — тон на „ти“, без финансов/правен съвет.

2. RAG vs fine-tuning — разлика и кога кое? RAG (Retrieval-Augmented Generation) — подаваш релевантни документи към prompt-а в реално време; знанието е външно и се обновява лесно, без преобучение. Fine-tuning — преобучаваш модела върху данни; променя поведение/стил, по-скъпо, не е за често-сменящи се факти. За обновяващ се knowledge base (видеа/текстове на МАБИ) → RAG. За устойчив тон/формат → fine-tuning. Често се комбинират — fine-tune за стила + RAG за фактите. Обикновено RAG е първият избор.

3. AI Ментор обучен с видеа и текстове — как структурираш knowledge base-a? Видеата на МАБИ вече се превръщат в текст с Whisper (във Video Streamer pipeline-a). Оттам: текстът се нарязва на по-малки части (chunks) → embeddings, пазени във vector store. При въпрос semantic search връща релевантните части в prompt-a (това е RAG). Добавям метаданни (източник, тема, дата) за филтриране и цитиране.

Пример: уебинар „Как да вдигнеш цените“ → Whisper изважда текста му → нарязва се на части по ~500 думи → всяка става embedding с метаданни (тема: ценообразуване). Член пита „как да вдигна цените?“ → въпросът става embedding → semantic search намира точно тези части → влизат в prompt-a → менторът отговаря и цитира урока.

4. Как тестваш дали AI дава качествени отговори? Подготвям eval set — набор от представителни въпроси с очаквани характеристики, плюс golden set (еталонни примери с известен правилен отговор). За всеки отговор проверявам: фактическа точност, спазване на тона и границите (без забранени съвети), релевантност. Комбинирам ръчен преглед с LLM-as-a-judge (друг модел оценява по зададени критерии — rubric). В production следя реалните отговори и събирам обратна връзка от потребителите.

JavaScript / Python

1. Интерактивен калкулатор в браузъра, без backend — как? Чист frontend, без сървър: state за входа, реактивно изчисление, веднага рендвам резултата. За запазване избирам според нуждата: sessionStorage ако трябва да се помни само по време на текущата сесия (напр. история на сметките), или localStorage ако трябва да оцелее след затваряне на браузъра.

```
function Calculator() {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  const [op, setOp] = useState("+");
  const [history, setHistory] = useState(
    () => JSON.parse(sessionStorage.getItem("history")) || "[]"
  );

  const calc = (x, y, o) => {
    o === "+" ? x + y :
    o === "-" ? x - y :
    o === "*" ? x * y :
    y !== 0 ? x / y : "деление на 0";
  };

  const result = calc(a, b, op);

  const save = () => {
    const next = [...history, `${a} ${op} ${b} = ${result}`];
    setHistory(next);
    sessionStorage.setItem("history", JSON.stringify(next)); // помни само в сесията
  };

  return (
    <div>
      <input type="number" value={a} onChange={(e) => setA(+e.target.value)} />
      <select value={op} onChange={(e) => setOp(e.target.value)}>
        <option>+</option><option>-</option><option>*</option><option>/</option>
      </select>
      <input type="number" value={b} onChange={(e) => setB(+e.target.value)} />
      <p>Резултат: {result}</p>
      <button onClick={save}>Запази</button>
      <ul>{history.map((h, i) => <li key={i}>{h}</li>)}</ul>
    </div>
  );
}
```

2. Как съхраняваш API ключове — какво НЕ трябва? НИКОГА: в кода, в Git, в frontend/браузъра (целият JS там е видим), в AI чатове/помощници, hardcoded, нито в логове. **Правилно:** тайните живеят само на backend, в environment променливи (файлът с реалните стойности е gitignored). Само server-side. Least privilege и ротация при съмнение за изтичане — сменя се стойността на ключа, не кодът. В production — secret manager (HashiCorp Vault, AWS Secrets Manager, Doppler, или secret store на платформата), не само .env файл. При нужда от по-голяма сигурност — различни ключове за всяка среда (dev / staging / prod), за да не компрометира теч в dev и prod. Важно: в чист React/frontend няма безопасно място за таен ключ — стига до браузъра; затова тайните минават през backend/proxy.

Скорост и самостоятелност

1. Когато имаш задача, но не ти е ясно всичко — питаш веднага или пробваш сам? Първо поглеждам проблема от страни, за да го разбера добре. После започвам да проучвам какви решения вече са прилагани за подобен проблем. Тук обаче внимавам за тънката граница: да не се задълбочавам безкрайно в търсене, ако не намеря верния подход бързо — да не изпадна в безкраен „ерог loop“ или да затъна с часове. Причината е срокът — съобразявам се с него. Ако не намеря готово решение, идвам с поне два-три възможни подхода, с конкретни въпроси по тях и предложение — и чак тогава питам. Тоест не питам с празни ръце, а с подготвена почва.

2. Дай пример за нещо, което е трябвало да се направи бързо без идеален план. Добър пример са семинарите в [rensa.club](#). Имаха краен срок април, а ги започнахме около 3 седмици преди това — без план и без ясна концепция. Целият проект беше замислен и одобрен от хора, които нямаха представа как точно трябва да изглежда: нямаше спецификация, нито изисквания — само няколко изречения „искаме видеа на живо“ и идеи от рода на „видях в една онлайн академия това и това, не става ли и при нас?“.

Тоест нямаш реален план, нито архитектура, и около 2 седмици да изградя нещо без ясни изисквания. Затова първо разгледах няколко академии — записах се в две, за да видя отвътре как са го направили и какви функционалности предлагат. После зададох ясен контекст и правила на Claude Code, с конкретни промптове и ограничения. Беше ми ясно, че без него няма как да напиша нещо толкова голямо и да остане време за тестване — под този срок това беше единственият начин да изляза навреме с работещ резултат. Накрая ги пуснахме навреме и работят в production.

3. Как приоритизираш когато имаш 5 задачи и всичко е "спешно"? Любопитно е, че самото число „5“ е показателно: около 5 неща е границата на работната памет (правилото на Милър) — точката, в която вече не можеш да държиш всичко в главата си и ти трябва система. Затова за мен същината тук не е „да работиш по-бързо“, а „да имаш система за подреждане“.

Моята система — приоритизирам по няколко критерия:

- **Блокиране** — кое спира други хора или процеси? Отпушвам го първо.
- **Срок** — кое има реален твърд deadline?
- **Стойност/impact** — кое носи най-голяма полза?
- **Бързи резултати** — има ли бързи неща, които веднага разчистват част от списъка?
- **Неизвестност** — задача, която още не разбирам напълно, крие риск; разучавам я рано, за да я де-рисувам, преди да взриви плана близо до срока.

„Спешно“ не значи „важно“ — разделям ги и комуникирам кое кога ще стане, вместо да обещавам всичко наведнъж. Точно така подходах и към това задание: първо задачите с конкретни deliverables и срок (A, B, C, D), а техническите въпроси — накрая.

Комуникация с non-tech хора

1. Как обясняваш технически проблем на нетехник? Пример. От две години работя с възрастни хора в [rensa.club](#) — цялата платформа е за дигитална грамотност, да им помогне да навлязат в дигиталния свят. Това ме научи много за обясняването на нетехнически хора.

Как го правя: нетехническите хора оценяват честен и прост отговор, който разбират — без жаргон. Често измислям пример от реалния живот, за да обясня нещо дигитално. Например, когато се страхуват „ще изтрия нещо важно“, им казвам, че е като кошчето за боклук вкъщи — дори да изхвърлиш нещо, можеш да го извадиш обратно, докато не изпразниш кошчето.

Но най-важното, което научих: при повечето водещият фактор е притеснението — „дали няма да объркам нещо" и „после поправимо ли е". Затова често започвам със успокоение: „спокойно, и да го объркаме, винаги можем да го оправим". Чак след това идва самото обяснение.

2. Виждаш, че нещо не е добра идея технически, но ръководителят настоява — какво правиш? Първо представям моята идея. Ако не се съгласи веднага, излагам ясно плюсовете — какво печелим — и реалните рискове на моето решение. После обсъждаме и неговия вариант, с плюсовете и минусите в него. Тоест не просто „не", а аргументиран разговор и за двете страни.

Ако след това реши да продължи с неговото — той е ръководителят и се съобразявам, изпълнявам го професионално. Но ако съм сигурен в решението си, наистина ще опитам да го убедя с аргументи — не от инат.